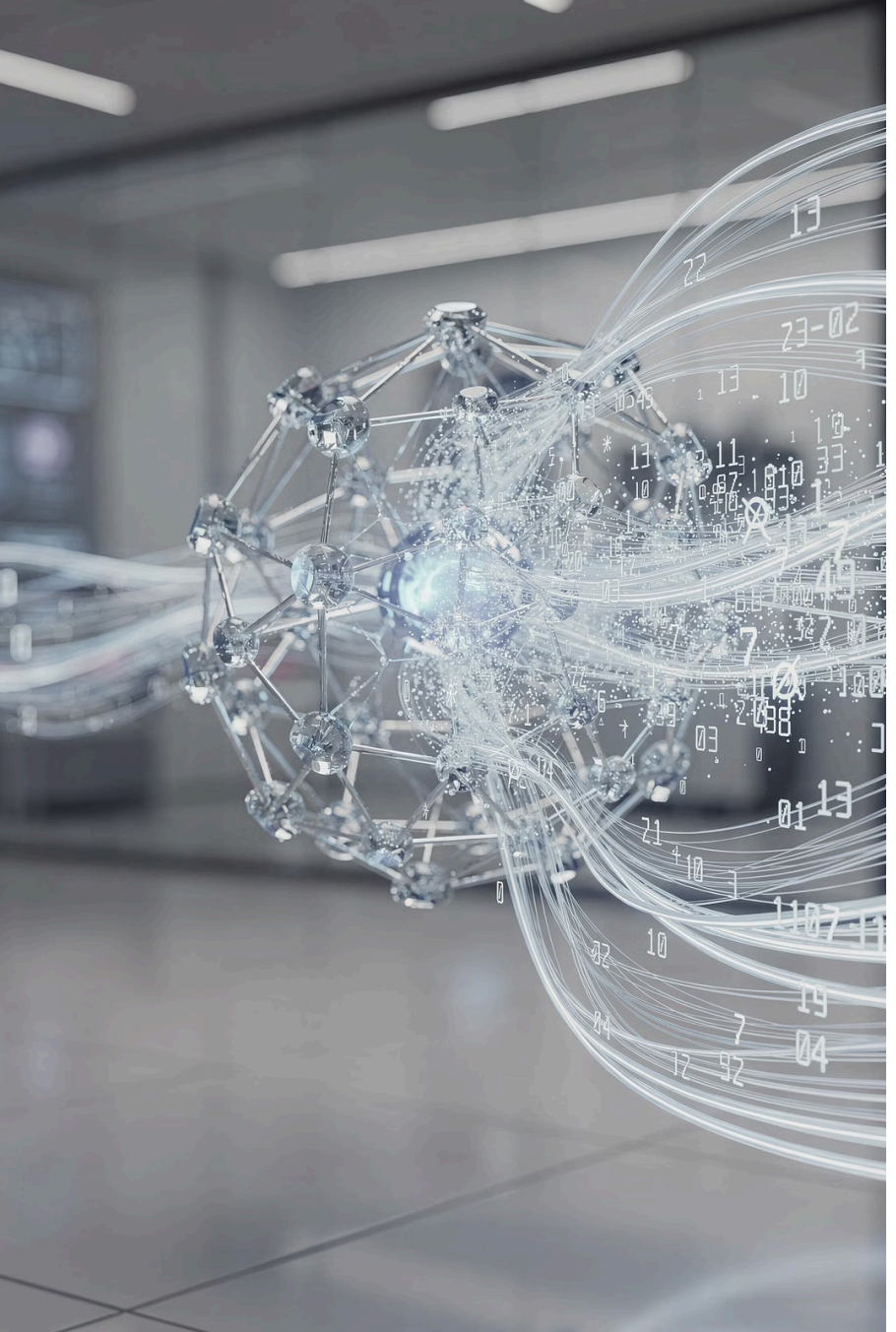
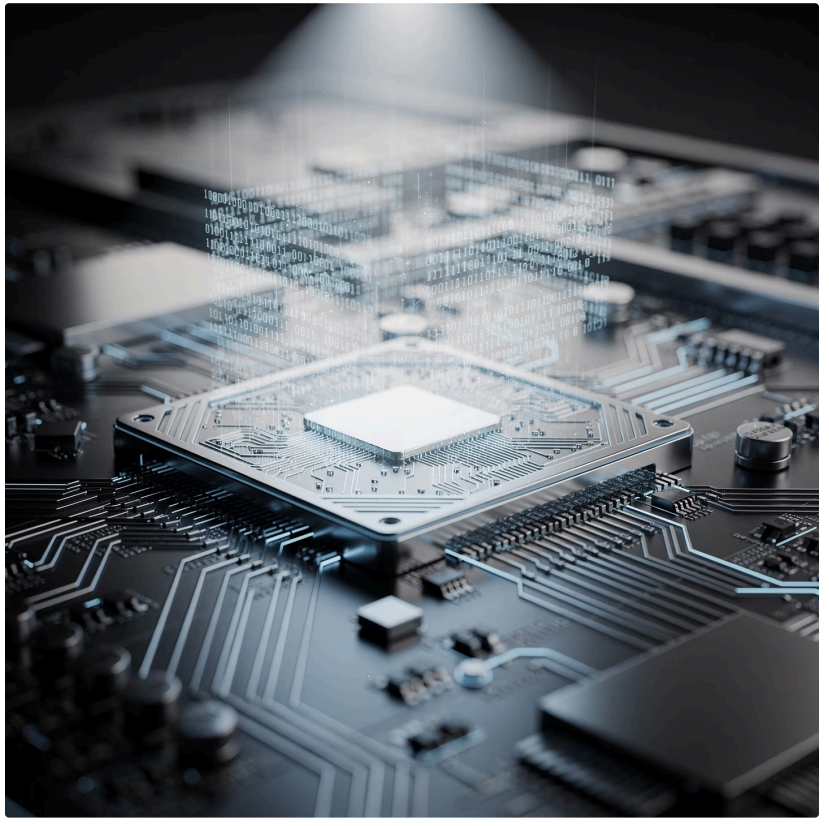


Vectorization Techniques in Natural Language Processing

Exploring the fundamental methods that transform human language into machine-readable numerical representations, enabling computers to process and understand text.

FOUNDATION

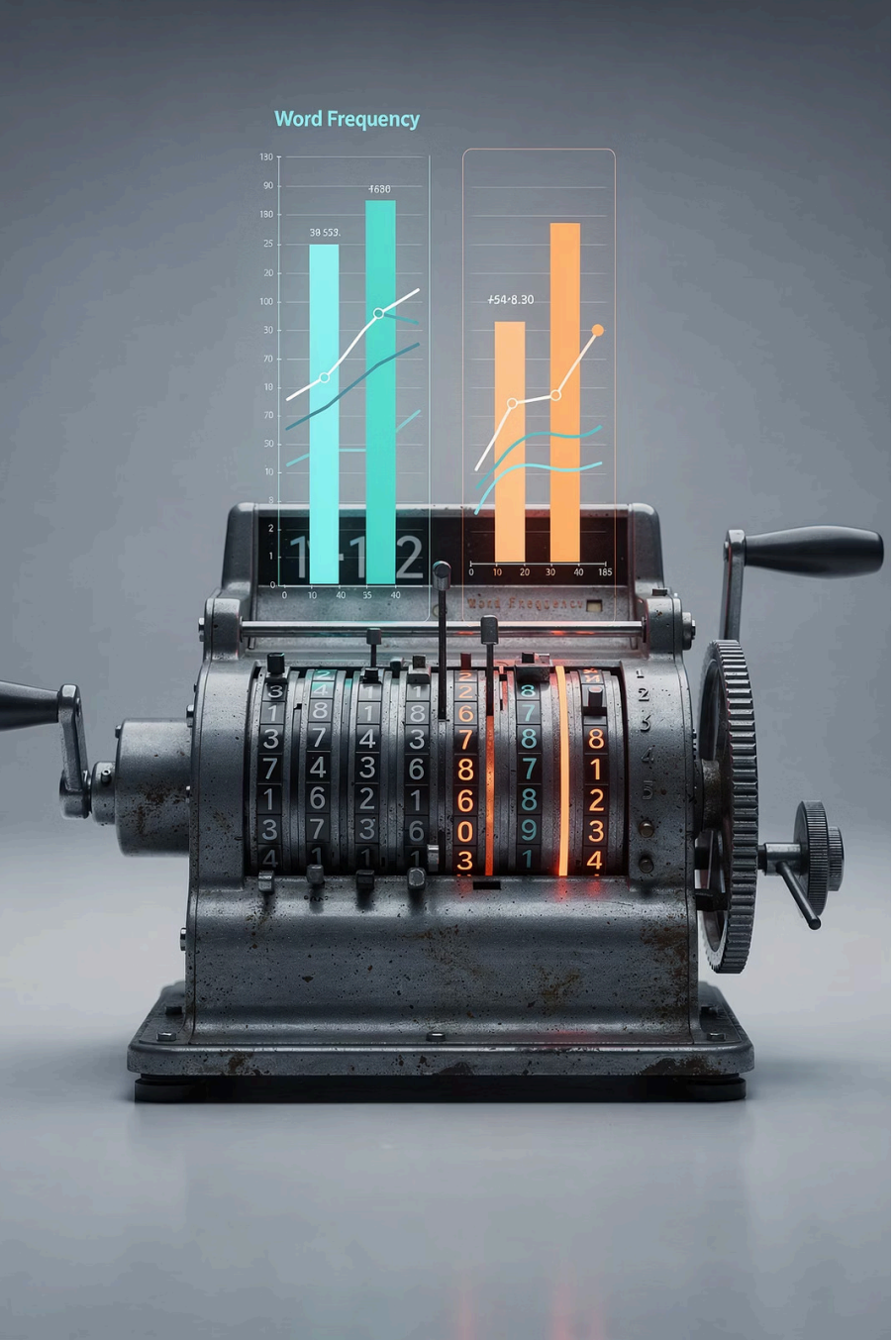
Why Vectorize Text?



At their core, computers operate on numbers, not words. Vectorization bridges this fundamental gap by converting text into numerical vectors that machine learning models can process and analyse.

This transformation is crucial because it enables algorithms to capture meaning, identify context, and understand complex relationships between words and documents. Without vectorization, the rich world of human language would remain inaccessible to computational analysis.

The choice of vectorization technique profoundly impacts model performance, making it essential to understand the strengths and trade-offs of different approaches.



Chapter 1

Classic Frequency-Based Methods

Before sophisticated neural networks transformed natural language processing, researchers developed elegant mathematical techniques based on word frequencies and statistical patterns. These foundational methods remain remarkably effective for many practical applications today.



TECHNIQUE

Bag of Words (BoW)

Word Counting

Represents text as a vector of word counts, tracking how many times each vocabulary word appears in a document.

Order Agnostic

Completely ignores word order and grammatical structure, treating text as a literal "bag" of independent words.

Fast & Effective

Simple to implement, computationally efficient, and surprisingly effective for text classification tasks.

BoW Example: Document Vector

Let's see how Bag of Words transforms a sentence into a numerical representation. This concrete example demonstrates the simplicity and limitations of the frequency-based approach.

01

Define Vocabulary

Establish the complete set of unique words we'll track: **[king, queen, throne, cat]**

02

Input Document

Our example sentence: **"king and queen on the throne"**

03

Count Occurrences

Tally how many times each vocabulary word appears in the document

04

Generate Vector

Final numerical representation: **[1, 1, 1, 0]** (counts for king, queen, throne, cat)

Notice how "and", "on", and "the" are ignored because they're not in our vocabulary, and "cat" receives a count of zero as it doesn't appear in the document.

ADVANCED

Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF represents a significant refinement over simple word counting, introducing the concept of word importance based on both local and global statistics.

Term Frequency (TF)

Measures how often a word appears within a single document, indicating local importance.

Inverse Document Frequency (IDF)

Downweights words that appear frequently across many documents in the corpus.

Combined Weight

Multiplies TF and IDF to highlight unique, meaningful words specific to each document.



TF-IDF Impact

Key Benefits

- Automatically reduces noise from common words like "the", "and", "is" that appear everywhere
- Dramatically improves relevance in search engines and document classification systems
- Requires no manual stop word lists or preprocessing rules
- Computationally efficient and interpretable

Practical Example

Consider the word **"pizza"** in two contexts:

Pizza Restaurant Review

High TF-IDF score because "pizza" appears frequently here but not in most other documents.

General News Corpus

Lower TF-IDF score because the word is diluted across many food-related articles.



Chapter 2

Word Embeddings — The Semantic Leap

The emergence of word embeddings revolutionised natural language processing by moving beyond simple frequency counts to capture deeper semantic meaning. These dense vector representations learn from context, enabling machines to understand that "king" relates to "queen" in ways that transcend mere co-occurrence statistics.

What Are Word Embeddings?



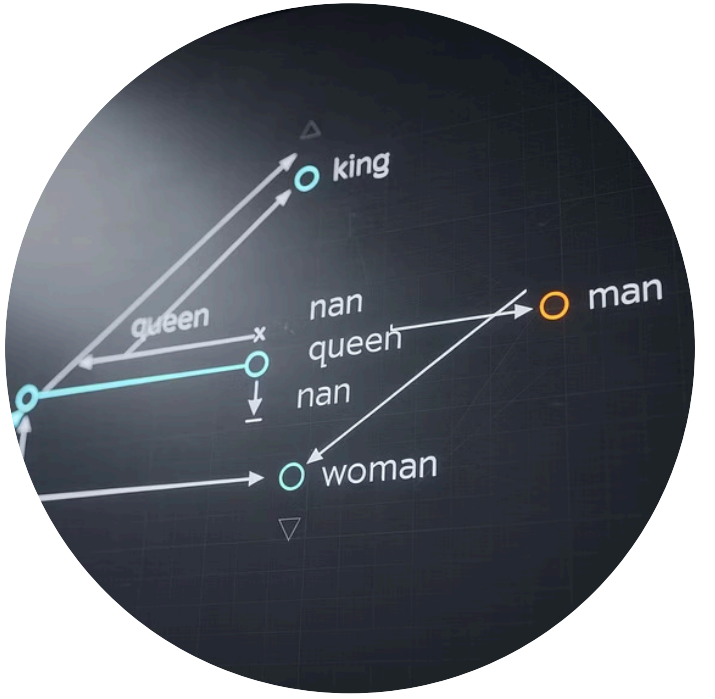
Dense Representations

Unlike sparse BoW vectors with mostly zeros, embeddings are dense vectors (typically 100-300 dimensions) where every value carries information.



Semantic Similarity

Words appearing in similar contexts receive similar vector representations, capturing meaning through distributional patterns.



Relationship Modelling

Enable vector arithmetic like "king - man + woman ≈ queen", capturing analogies and linguistic relationships.

GOOGLE 2013

Word2Vec: Predicting Context

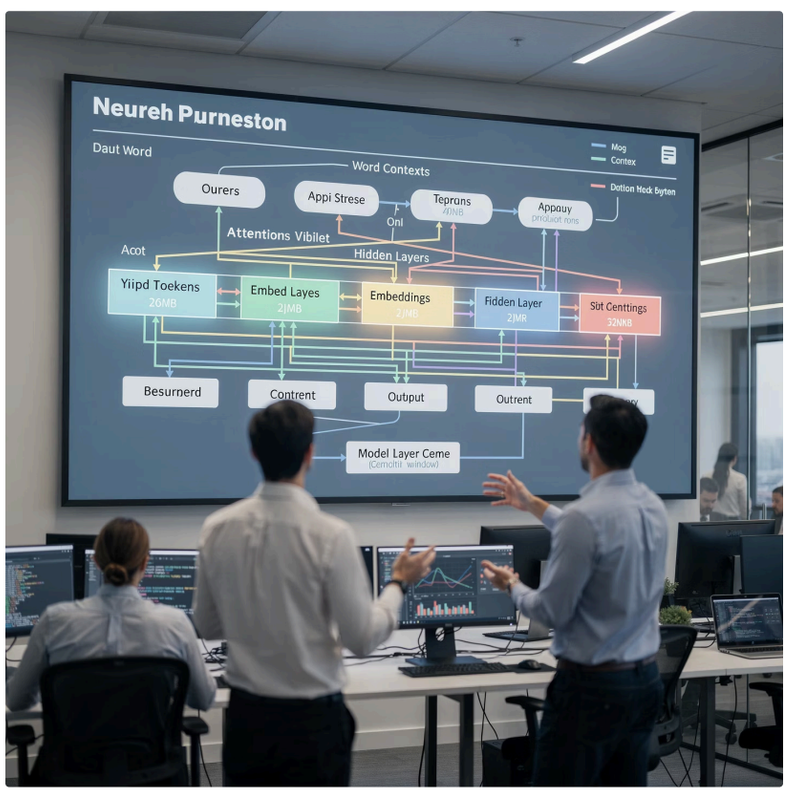
Developed by Tomas Mikolov and colleagues at Google in 2013, Word2Vec introduced an efficient neural network approach to learning word embeddings by predicting contextual relationships.

CBOW Architecture

Continuous Bag of Words predicts a target word from its surrounding context words. Faster to train, better for smaller datasets.

Skip-Gram Architecture

Predicts context words from a target word. Slower but works better with rare words and smaller training corpora.



The model learns by adjusting vectors to maximise prediction accuracy. Through this process, words like "king", "queen", and "throne" naturally cluster together because they appear in similar linguistic contexts throughout the training corpus.

STANFORD

GloVe: Global Co-occurrence Statistics

Global Vectors for Word Representation (GloVe), developed by researchers at Stanford University, takes a different philosophical approach to learning embeddings by explicitly leveraging corpus-wide co-occurrence statistics.

1

Construct Matrix

Build a word co-occurrence matrix counting how often word pairs appear together across the entire corpus.

2

Matrix Factorization

Apply mathematical factorization techniques to decompose this sparse matrix into dense vector representations.

3

Capture Patterns

Resulting embeddings preserve global statistical patterns, capturing nuanced semantic and syntactic relationships.

GloVe's global approach often produces embeddings that excel at analogical reasoning tasks, such as determining that "Paris is to France as London is to England".



Word2Vec vs GloVe

1 Word2Vec

- **Local context windows:** Learns from surrounding words in fixed-size windows
- **Incremental training:** Can be updated efficiently as new data arrives
- **Memory efficient:** Doesn't require storing full co-occurrence matrix
- **Fast training:** Particularly efficient with Skip-Gram negative sampling

1 GloVe

- **Global statistics:** Leverages corpus-wide co-occurrence patterns
- **Analogy performance:** Often superior at capturing word relationships
- **Deterministic:** Same corpus produces identical embeddings
- **Mathematical elegance:** Clear objective function based on ratios

The choice between Word2Vec and GloVe depends on your specific task requirements, available computational resources, and the characteristics of your dataset. Many practitioners experiment with both approaches to determine which yields superior performance for their particular application.

Visualising Word Embeddings

To understand high-dimensional word embeddings intuitively, we can project them into 2D space using dimensionality reduction techniques like t-SNE or PCA. These visualizations reveal the semantic structure captured by the embeddings.

Royalty Cluster

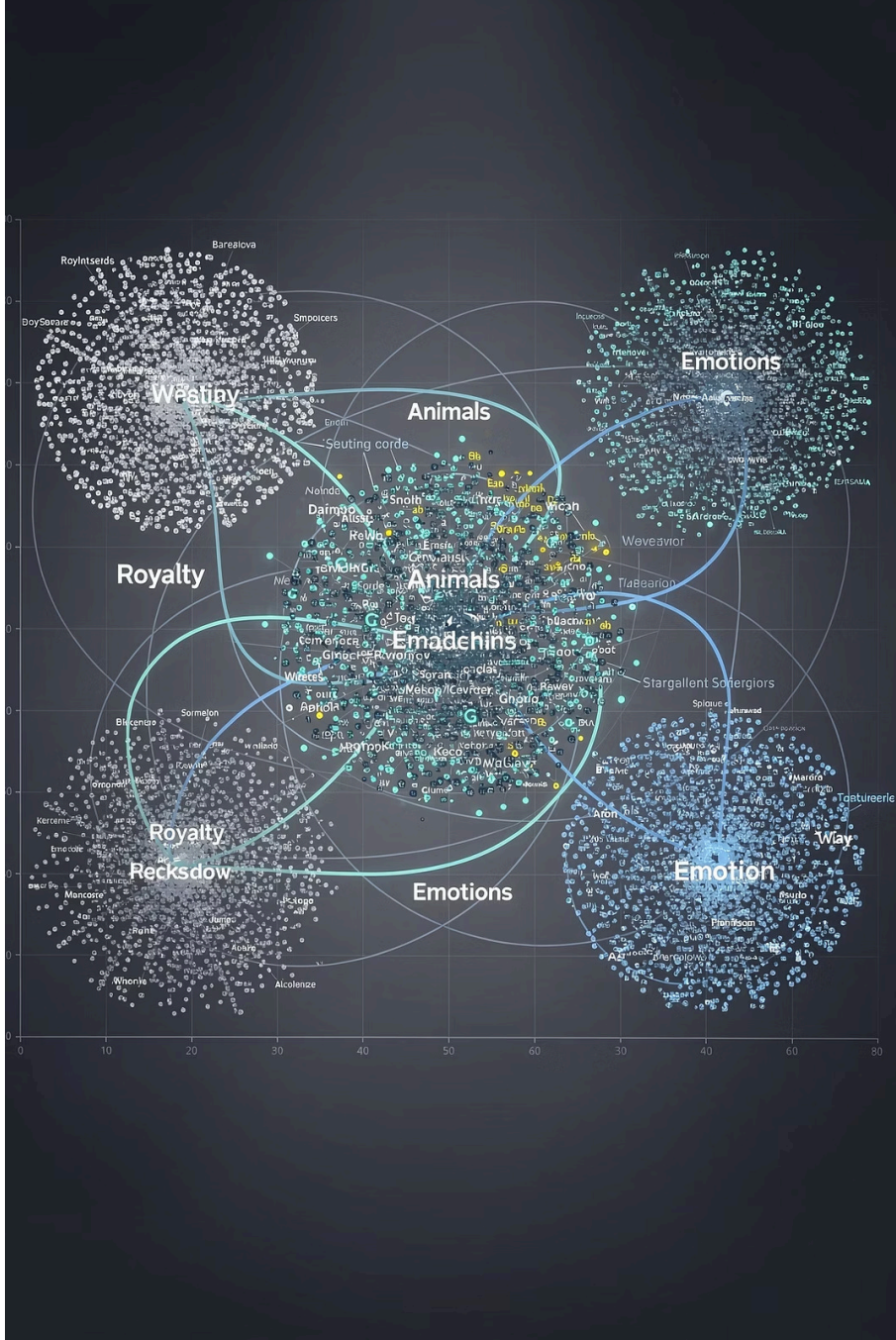
Words like "king", "queen", "prince", "throne", and "crown" cluster together, demonstrating captured hierarchical and familial relationships.

Animals Cluster

Animal terms group by characteristics: "dog", "cat", "wolf" near each other; "eagle", "hawk", "sparrow" form their own subcluster.

Emotions Cluster

Emotional words reveal sentiment structure: "happy", "joyful", "delighted" versus "sad", "melancholy", "depressed" occupy opposite regions.



Conclusion: Choosing Your Vectorization Technique

The landscape of vectorization techniques offers diverse tools, each with distinct strengths suited to different natural language processing challenges. Understanding these trade-offs enables informed architectural decisions.

1 Bag of Words & TF-IDF

Best for: Text classification, information retrieval, document similarity when interpretability matters

- Simple to implement and understand
- Computationally efficient for large corpora
- Highly interpretable feature weights
- Excellent baseline performance

2 Word Embeddings (Word2Vec, GloVe)

Best for: Sentiment analysis, machine translation, question answering, any task requiring semantic understanding

- Capture deeper contextual meaning
- Enable transfer learning from large corpora
- Support analogical reasoning
- Essential for modern deep learning NLP

As you design natural language processing systems, consider starting with simpler frequency-based methods to establish baselines, then progress to word embeddings when semantic understanding becomes crucial. The most sophisticated approach isn't always the best—match your technique to your specific task requirements, available data, and computational constraints.